

CS102: Operating Systems Architecture & Resource Management

Department of Computer Science & Engineering | Academic Handout

Page 1: Process Lifecycle, Threads & CPU Scheduling

2.1 Process Lifecycle and Process Control Block (PCB)

A process is a program in execution, consisting of program code (text segment), program counter, stack (temporary local variables and return addresses), data segment (global variables), and heap (dynamically allocated memory).

The operating system maintains a Process Control Block (PCB) for each process containing: Process ID (PID), Process State, CPU registers, CPU scheduling priority, memory-management information (page tables), and I/O status information.

Process states transition through: New -> Ready -> Running -> Waiting (Blocked) -> Terminated.

A Context Switch occurs when the CPU scheduler switches execution from one process to another. The OS saves the current state in the active PCB and restores the saved state of the scheduled PCB. Context switching represents pure system overhead because no useful user computation occurs during the transition.

Threads (lightweight processes) within the same process share code, data, open files, and address space, but each thread retains its own thread ID, program counter, register set, and private stack.

2.2 CPU Scheduling Algorithms

- First-Come, First-Served (FCFS): Non-preemptive. Suffers from the Convoy Effect where short CPU-burst processes wait behind a long CPU-burst process, severely degrading average turnaround time.
- Shortest Job First (SJF) & Shortest Remaining Time First (SRTF): Preemptive (SRTF) or non-preemptive (SJF). Provably optimal in minimizing average waiting time, but susceptible to process starvation if short jobs continuously arrive.
- Round Robin (RR): Preemptive scheduling designed for time-sharing systems. Each process is allocated a fixed time quantum q . If the process does not complete within q , it is preempted and returned to the tail of the ready queue. If q is excessively large, RR degenerates into FCFS; if q is too small, context-switch overhead dominates.

Page 2: Virtual Memory, Paging Architecture & Replacement Algorithms

2.3 Virtual Memory and Paging

Virtual memory decouples user logical address space from physical memory. The Memory Management Unit (MMU) translates logical addresses composed of a page number (p) and offset (d) into physical frame numbers (f) using a per-process Page Table.

To accelerate address translation, CPUs employ a high-speed associative hardware cache called the Translation Lookaside Buffer (TLB).

Effective Memory Access Time (EMAT) is computed as: $EMAT = h * (t_{tlb} + t_{mem}) + (1 - h) * (t_{tlb} + 2 * t_{mem})$, where h is the TLB hit ratio, t_{tlb} is TLB access latency, and t_{mem} is main memory access latency.

A Page Fault exception occurs when a referenced page is marked invalid (not present in physical RAM). The OS traps to kernel mode, initiates disk I/O to swap in the requested page from backing store, updates the page table entry, and restarts the faulting instruction.

2.4 Page Replacement Algorithms

When physical memory is exhausted and a page fault occurs, the OS must select a victim frame to evict using a page replacement policy:

- First-In, First-Out (FIFO): Evicts the oldest loaded page. FIFO is vulnerable to Belady's Anomaly, where increasing the number of allocated physical frames leads to an increase in the total number of page faults.
- Optimal Page Replacement (OPT / MIN): Evicts the page that will not be used for the longest future duration. OPT cannot be implemented in general-purpose OS kernels because future memory reference strings are unknown; it serves as a theoretical performance upper bound.
- Least Recently Used (LRU): Evicts the page that has not been referenced for the longest past interval. LRU is a stack algorithm and is mathematically immune to Belady's Anomaly.
- Clock Algorithm (Second Chance): Practical approximation of LRU using a single reference bit per frame and a rotating circular pointer.

Page 3: Deadlocks, Coffman Conditions & Concurrency Synchronization

2.5 Deadlocks and the Four Coffman Conditions

DEFINITION: A deadlock is a state where a set of processes are blocked because each process holds a resource and waits for another resource held by another process in the set. A deadlock can arise if and only if ALL FOUR of the following Coffman Conditions hold simultaneously:

1. Mutual Exclusion: At least one resource must be held in a non-shareable mode (only one process can use it at any instant).
2. Hold and Wait: A process must currently hold at least one resource and simultaneously request additional resources held by other processes.
3. No Preemption: Resources cannot be forcibly seized; a resource can only be released voluntarily by the process holding it after completing its task.
4. Circular Wait: A closed chain of processes $\{P_0, P_1, \dots, P_n\}$ exists such that P_0 is waiting for a resource held by P_1 , P_1 is waiting for P_2 , and P_n is waiting for a resource held by P_0 .

Deadlock Handling Approaches: Prevention (systematically invalidating at least one of the four Coffman conditions), Avoidance (using Dijkstra's Banker's Algorithm to verify that state transitions remain safe before granting requests), and Detection & Recovery (constructing Resource Allocation Graphs to detect cycles and terminating victim processes).

2.6 Synchronization Primitives: Semaphores and Mutexes

A Mutex is a locking mechanism used to synchronize access to a critical section, permitting only one thread of ownership at a time.

A Semaphore is a signaling integer variable S accessed via two atomic operations: `wait()` [decrement S , block if $S \leq 0$] and `signal()` [increment S , unblock a waiting process]. Counting semaphores coordinate access to a finite pool of identical resource instances, while binary semaphores behave similarly to mutexes.